

# Assignment 3-Yue CHENGHAO-227154

## code

```
import java.util.HashSet;
import java.util.Scanner;
import java.util.Set;

public class BSTApp {

    // ----- Node -----
    static class Node {
        int data;
        Node left, right;

        Node(int data) {
            this.data = data;
        }
    }

    // ----- BST -----
    static class BST {
        private Node root;

        // Insert (no duplicates)
        public void insert(int value) {
            root = insert(root, value);
        }

        private Node insert(Node node, int value) {
            if (node == null) return new Node(value);

            if (value < node.data) {
                node.left = insert(node.left, value);
            } else if (value > node.data) {
                node.right = insert(node.right, value);
            } else {
                // duplicate
                System.out.println("ERROR: Duplicate value. Not inserted.");
            }
            return node;
        }
    }
}
```

```

// Search
public boolean search(int value) {
    return search(root, value);
}

private boolean search(Node node, int value) {
    if (node == null) return false;
    if (node.data == value) return true;
    return (value < node.data) ? search(node.left, value) :
search(node.right, value);
}

// Delete (returns true if deleted, false if not found)
public boolean delete(int value) {
    if (!search(value)) return false;
    root = delete(root, value);
    return true;
}

private Node delete(Node node, int value) {
    if (node == null) return null;

    if (value < node.data) {
        node.left = delete(node.left, value);
    } else if (value > node.data) {
        node.right = delete(node.right, value);
    } else {
        // found node to delete
        if (node.left == null) return node.right;
        if (node.right == null) return node.left;

        Node min = findMin(node.right);
        node.data = min.data;
        node.right = delete(node.right, min.data);
    }
    return node;
}

private Node findMin(Node node) {
    Node cur = node;
    while (cur.left != null) cur = cur.left;
    return cur;
}

// Height: empty = -1, single node = 0
public int height() {
    return height(root);
}

```

```

private int height(Node node) {
    if (node == null) return -1;
    return 1 + Math.max(height(node.left), height(node.right));
}

// Balanced: for every node, |lh-rh| <= 1
public boolean isBalanced() {
    return isBalanced(root);
}

private boolean isBalanced(Node node) {
    if (node == null) return true;
    int lh = height(node.left);
    int rh = height(node.right);
    if (Math.abs(lh - rh) > 1) return false;
    return isBalanced(node.left) && isBalanced(node.right);
}

// Traversals (comma-separated)
public String preorderNLR() {
    StringBuilder sb = new StringBuilder();
    preorder(root, sb);
    return sb.toString();
}

public String inorderLNR() {
    StringBuilder sb = new StringBuilder();
    inorder(root, sb);
    return sb.toString();
}

public String postorderLRN() {
    StringBuilder sb = new StringBuilder();
    postorder(root, sb);
    return sb.toString();
}

private void preorder(Node node, StringBuilder sb) {
    if (node == null) return;
    appendCommaSeparated(sb, node.data);
    preorder(node.left, sb);
    preorder(node.right, sb);
}

private void inorder(Node node, StringBuilder sb) {
    if (node == null) return;
    inorder(node.left, sb);

```

```

        appendCommaSeparated(sb, node.data);
        inorder(node.right, sb);
    }

    private void postorder(Node node, StringBuilder sb) {
        if (node == null) return;
        postorder(node.left, sb);
        postorder(node.right, sb);
        appendCommaSeparated(sb, node.data);
    }

    private void appendCommaSeparated(StringBuilder sb, int value) {
        if (sb.length() > 0) sb.append(",");
        sb.append(value);
    }

    // Display X command output
    public void displayInfo() {
        System.out.println("NLR: " + preorderNLR());
        System.out.println("LNR: " + inorderLNR());
        System.out.println("LRN: " + postorderLRN());
        System.out.println("HEIGHT: " + height());
        System.out.println("BALANCED: " + (isBalanced() ? "YES" : "NO"));
    }
}

// ----- Help -----
private static void showHelp() {
    System.out.println("Valid commands:");
    System.out.println(" <positive number> : insert into BST (no
duplicates)");
    System.out.println(" s <number> : search for number");
    System.out.println(" d <number> : delete number if found");
    System.out.println(" X : display traversals, height,
balanced");
    System.out.println(" ? : show this help");
    System.out.println(" Q : quit");
}

// ----- Main -----
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    BST tree = new BST();

    // Optional: track duplicates quickly (not required, BST already blocks
duplicates)
    Set<Integer> seen = new HashSet<>();

```

```

showHelp();
while (true) {
    System.out.print("> ");
    String input = sc.nextLine().trim();

    if (input.equalsIgnoreCase("Q")) {
        System.out.println("Bye.");
        break;
    }

    if (input.equals("?")) {
        showHelp();
        continue;
    }

    if (input.equalsIgnoreCase("X")) {
        tree.displayInfo();
        continue;
    }

    if (input.startsWith("s ") || input.startsWith("S ")) {
        String[] parts = input.split("\\s+");
        if (parts.length != 2) {
            System.out.println("ERROR: Invalid search command. Type ?
for help.");
            continue;
        }
        Integer val = parsePositiveInt(parts[1]);
        if (val == null) {
            System.out.println("ERROR: Invalid number. Must be a
positive integer.");
            continue;
        }
        System.out.println(tree.search(val) ? "FOUND" : "NOT FOUND");
        continue;
    }

    if (input.startsWith("d ") || input.startsWith("D ")) {
        String[] parts = input.split("\\s+");
        if (parts.length != 2) {
            System.out.println("ERROR: Invalid delete command. Type ?
for help.");
            continue;
        }
        Integer val = parsePositiveInt(parts[1]);
        if (val == null) {
            System.out.println("ERROR: Invalid number. Must be a
positive integer.");

```

```

        continue;
    }
    boolean deleted = tree.delete(val);
    if (deleted) {
        System.out.println("DELETED");
        // keep seen consistent if you're using it
        seen.remove(val);
    } else {
        System.out.println("NOT FOUND");
    }
    continue;
}

// Otherwise treat as insertion attempt
Integer val = parsePositiveInt(input);
if (val == null) {
    System.out.println("ERROR: Invalid command. Type ? for help.");
    continue;
}

if (seen.contains(val)) {
    System.out.println("ERROR: Duplicate value. Not inserted.");
    continue;
}

tree.insert(val);
seen.add(val);
System.out.println("INSERTED");
}

sc.close();
}

private static Integer parsePositiveInt(String s) {
    try {
        int v = Integer.parseInt(s);
        if (v <= 0) return null;
        return v;
    } catch (NumberFormatException e) {
        return null;
    }
}
}

```

- output

```

○ > cd "/Users/y/Desktop/test/" && javac BSTApp.java && java BSTApp
Valid commands:
<positive number> : insert into BST (no duplicates)
s <number>         : search for number
d <number>         : delete number if found
X                 : display traversals, height, balanced
?                 : show this help
Q                 : quit
>

```

```

○ > cd "/Users/y/Desktop/test/" && javac BSTApp.java && java BSTApp
Valid commands:
<positive number> : insert into BST (no duplicates)
s <number>         : search for number
d <number>         : delete number if found
X                 : display traversals, height, balanced
?                 : show this help
Q                 : quit
> 7
INSERTED
> 6
INSERTED
> 5
INSERTED
> 4
INSERTED
> 3
INSERTED
> 2
INSERTED
> 1
INSERTED
> x
NLR: 7,6,5,4,3,2,1
LNR: 1,2,3,4,5,6,7
LRN: 1,2,3,4,5,6,7
HEIGHT: 6
BALANCED: NO
> s 5
FOUND
> s 8
NOT FOUND
> x 5
ERROR: Invalid command. Type ? for help.
> d 5
DELETED
> x
NLR: 7,6,4,3,2,1
LNR: 1,2,3,4,6,7
LRN: 1,2,3,4,6,7
HEIGHT: 5
BALANCED: NO
> █

```